CityVision · Preuve de concept · Texte de la présentation

Durée cible : 20 minutes. Plan prévisionnel ≈ 4 min 20, démarche et résultats ≈ 9 min 20, démonstration et conclusion ≈ 5 min, marge ≈ 1 min 20. Rythme de lecture : environ 130 mots par minute. Le minutage cumulé est indiqué à la fin de chaque diapositive.

Partie 1 · Plan prévisionnel (≈ 4 min 20)

1. Segmenter la ville avec EoMT **0:20**

Bonjour Karim. Je vais présenter la preuve de concept menée sur CityVision, en trois temps. D'abord le plan prévisionnel : le jeu de données, le modèle choisi et les sources. Ensuite la démarche : les concepts du nouveau modèle et ses résultats face à la baseline. Enfin, une démonstration du dashboard déployé sur le cloud.

Cumul : 0:20

2. Point de départ : 0,816 de mIoU **0:50**

CityVision segmente des photos de rue en neuf classes : route, bâtiment, végétation, ciel, personne, voiture, panneau, vélo, et une classe fond pour tout le reste.

Le modèle actuel est un ResNet50-UNet. Il attribue une classe à chaque pixel, et il est servi par l'API et l'application du projet. Sur les 500 images de validation, il obtient 0,816 de mIoU. La mIoU, c'est la moyenne, sur les neuf classes, de l'intersection divisée par l'union entre la prédiction et la vérité terrain.

La question de cette preuve de concept est simple : un modèle récent, de moins de cinq ans, fait-il mieux, dans exactement les mêmes conditions ?

Cumul : 1:10

3. Cityscapes, ramené à 9 classes **1:00**

Le jeu de données est celui du projet : Cityscapes. Ce sont des photos prises depuis une voiture, dans des villes allemandes, annotées pixel par pixel. Il y a 2 975 images d'entraînement, de 18 villes, et 500 images de validation, de 3 autres villes. On travaille en 512 par 1024, soit la moitié de la résolution d'origine.

Le point important est le déséquilibre. Route, fond et bâtiments couvrent les trois quarts des pixels. Personnes, panneaux et vélos, en orange, font moins de 2 % à eux trois. Ce sont pourtant les objets critiques pour la conduite. Et les panneaux sont présents dans 94 % des images : ce ne sont pas des objets rares, ce sont de petits objets.

C'est donc sur ces petites classes que j'attends le gain d'un nouveau modèle.

Cumul : 2:10

4. Modèle retenu : EoMT (CVPR 2025) **1:20**

Le modèle retenu est EoMT, publié en mars 2025 et présenté à CVPR 2025. Quatre arguments.

Un : il change de paradigme. Au lieu de classer chaque pixel, il prédit un ensemble de masques, chacun avec sa classe. C'est l'approche qui domine aujourd'hui les classements de segmentation.

Deux : un masque couvre un objet entier. On attend donc un gain sur les petits objets, justement les classes faibles de la baseline.

Trois : il est simple et rapide. EoMT montre qu'un Vision Transformer seul suffit, sans les décodeurs des modèles précédents. Les auteurs annoncent une précision comparable à l'état de l'art, jusqu'à quatre fois plus vite.

Quatre : c'est faisable. Le code officiel est sous licence MIT, intégré à Hugging Face, avec des poids pré-entraînés.

Comme étape intermédiaire, j'ai aussi entraîné Mask2Former, de 2022, dont EoMT est une simplification. Cela permet de séparer ce qu'apporte le paradigme des masques, et ce qu'apporte EoMT lui-même.

Cumul : 3:30

5. Trois références de recherche **0:50**

Mes trois sources sont des articles de recherche publiés sur arXiv.

L'article EoMT, de Kerssies et collègues, 2025 : c'est l'algorithme mis en œuvre.

L'article Mask2Former, de Cheng et collègues, 2022 : il introduit la classification de masques et l'attention masquée, dont EoMT dérive.

Et l'article Cityscapes, de Cordts et collègues, 2016, pour le jeu de données et son protocole d'évaluation.

J'ai réutilisé, en le citant, le code de Hugging Face Transformers et la recette d'entraînement du dépôt officiel d'EoMT. Le reste est propre au projet : les neuf classes, la résolution, le pipeline de données, l'évaluation et le dashboard.

Cumul : 4:20

Partie 2 · Démarche et résultats (≈ 9 min 20)

6. Une comparaison équitable **0:40**

Pour que la comparaison soit honnête, tout ce qui peut être identique l'est : mêmes images, même résolution, même échantillonneur pondéré pour les classes rares, même budget de 20 époques avec arrêt anticipé, et même métrique, calculée par le même code. Aucun modèle n'a vu Cityscapes avant.

Ce qui diffère fait partie de chaque méthode : son pré-entraînement et la recette de son article. Mon GPU portable a 8 Go : les transformers sont entraînés par lots de 2, avec accumulation de gradient, ce qui donne le même lot effectif de 4 que la baseline.

Cumul : 5:00

7. Des pixels à un ensemble de masques 0:50

Premier concept : la formulation du problème.

La baseline produit neuf cartes de scores, et prend pour chaque pixel la classe de score maximal. Cela fait plus d'un demi-million de décisions indépendantes par image.

Mask2Former et EoMT prédisent un ensemble fixe de N paires : 100 pour Mask2Former, 200 pour EoMT. Chaque paire, c'est un vecteur de probabilités sur les classes, avec une classe « aucun objet », et un masque. Pour revenir à une carte par pixel, on additionne, pour chaque classe, les masques pondérés par leur probabilité, et on prend le maximum.

Côté données, chaque annotation devient un ensemble de masques binaires, un par classe présente. Voici les huit masques d'une image de validation.

Cumul : 5:50

8. Appariement hongrois, puis perte 0:50

Deuxième concept : comment apprendre un ensemble, alors que l'ordre des requêtes est arbitraire ? Par un appariement.

Pour chaque image, on calcule le coût d'associer chaque masque réel à chaque requête : la probabilité de la bonne classe, plus l'erreur sur le masque. L'algorithme hongrois trouve l'association de coût total minimal. Dans l'exemple, la requête 2 prend la route, la 5 la voiture, la 1 la personne. Les autres doivent prédire « aucun objet ».

La perte est ensuite calculée sur cet appariement, avec les poids de l'article : 2 pour la classe, 5 pour l'entropie binaire et 5 pour le Dice. Et elle est appliquée à chaque étape intermédiaire : c'est la supervision profonde.

Cumul : 6:40

9. L'attention masquée 0:40

Troisième concept, propre à Mask2Former : l'attention masquée.

Dans une attention classique, chaque requête regarde toute l'image. Ici, on ajoute un masque à l'attention : zéro là où le masque prédit à la couche précédente dépasse 0,5, moins l'infini ailleurs. Après le softmax, la requête ne regarde que sa propre zone, en orange sur le schéma. Le masque s'affine de couche en couche, et l'apprentissage converge plus vite.

Autour de cela, Mask2Former empile trois blocs : un backbone Swin, un pixel decoder et un décodeur de neuf couches. C'est précis, mais lourd.

Cumul : 7:20

10. Un ViT encodeur seul, sans décodeur 0:50

Quatrième concept : EoMT supprime tout ce que Mask2Former ajoute autour du backbone.

L'image est découpée en 2 048 patchs de 16 pixels et passe dans un ViT-B, pré-entraîné avec DINOv2. Avant le dixième bloc, on ajoute 200 requêtes. Dans les trois derniers blocs, requêtes et patchs se mélangent par simple auto-attention. Un petit module produit ensuite la classe et le masque de chaque requête.

Il est plus rapide pour deux raisons : il n'utilise que des opérations de ViT, très optimisées en demi-précision, et il ne calcule aucun masque intermédiaire à l'inférence.

Une adaptation est propre au projet : le modèle pré-entraîné attend des images carrées de 640 pixels. J'ai interpolé ses position embeddings vers notre grille de 32 par 64.

Cumul : 8:10

11. Mask annealing : retirer les masques — 0:40

Cinquième concept, celui qui rend EoMT rapide : le mask annealing.

Au début de l'entraînement, les trois derniers blocs utilisent l'attention masquée, qui aide à apprendre. Puis, bloc par bloc, la probabilité de l'appliquer décroît de 1 à 0. Sur mes 20 époques, le bloc 10 la perd entre les époques 4 et 8, le bloc 11 entre 8 et 12, le bloc 12 entre 12 et 16. Les quatre dernières époques sont entièrement sans masque.

J'ai validé à chaque époque sans aucun masque, exactement comme en production. Et la mIoU a continué de monter pendant le retrait, de 0,835 à 0,842.

Cumul : 8:50

12. Recette d'entraînement — 0:30

Le tableau résume les recettes. Chaque modèle garde celle de son article, pour ne pas le désavantager.

Deux points pour EoMT. Le taux d'apprentissage décroît par couche, d'un facteur 0,8 par bloc : les premières couches, déjà bien pré-entraînées, bougent peu. Et le warm-up se fait en deux temps : d'abord les têtes, avec le ViT gelé, puis le ViT.

Cumul : 9:20

13. EoMT : +0,027 de mIoU — 0:45

Le résultat principal. Sur les 500 images de validation, la baseline obtient 0,816 de mIoU, Mask2Former 0,842, et EoMT 0,843.

Les deux nouvelles méthodes gagnent environ 2,7 points. Pour mesurer ce que cela représente, il faut regarder l'erreur restante, un moins la mIoU. La baseline laisse 0,184 d'erreur, EoMT 0,157. EoMT supprime donc 15 % de l'erreur restante. Et davantage sur les classes difficiles : 29 % sur le fond.

À ce niveau, c'est important : la baseline avait déjà été retenue parmi cinq architectures CNN.

Cumul : 10:05

14. Gains sur les classes difficiles — 0:40

Le détail par classe confirme l'hypothèse du plan prévisionnel. Les gains les plus nets sont sur le fond, qui regroupe des éléments très variés : plus 0,071. Puis sur les personnes : plus 0,042. Et sur les panneaux et les vélos : plus 0,027. Ce sont les classes où la baseline était faible.

Huit classes sur neuf progressent. La seule baisse est le ciel, une grande zone simple, déjà très bien traitée. On note aussi que Mask2Former reste un peu meilleur qu'EoMT sur les panneaux.

Cumul : 10:45

15. Même précision, 2,4 × plus rapide 0:35

Côté vitesse, sur mon GPU en demi-précision : 19 millisecondes par image pour la baseline, 106 pour Mask2Former, 43 pour EoMT.

EoMT atteint la précision de Mask2Former en étant 2,4 fois plus rapide. Cela confirme l'argument de l'article, sur un autre jeu et une autre résolution.

Deux nuances. La baseline reste deux fois plus rapide qu'EoMT. Et sur CPU, comme sur le dashboard gratuit, l'avantage disparaît : il repose sur les cœurs tensoriels du GPU.

Cumul : 11:20

16. Des écarts réels et reproductibles 0:40

Deux vérifications de fiabilité.

À gauche, le bootstrap apparié : on ré-échantillonne 2 000 fois les images de validation, et on recalcule l'écart à chaque fois. L'intervalle à 95 % de l'écart avec la baseline est entièrement positif, pour Mask2Former comme pour EoMT : le gain ne dépend pas des images choisies. Entre EoMT et Mask2Former, l'intervalle contient zéro : c'est une égalité en précision.

À droite, la reproductibilité : j'ai ré-entraîné les trois modèles de zéro, dans le notebook livré. Les résultats diffèrent de moins de 0,002 des entraînements de référence.

Cumul : 12:00

17. Contours plus nets, moins de bruit 0:25

Trois exemples tirés du notebook. Sur la première image, la baseline produit des taches de fond parasites au bord de la route ; les deux transformers rendent des contours plus propres. Sur une seule image, le classement peut s'inverser : c'est pourquoi les conclusions reposent sur les 500 images et sur le bootstrap.

Cumul : 12:25

18. Limites de la méthode, pistes d'amélioration 0:45

Les limites de la méthode elle-même.

EoMT supprime le décodeur et repose entièrement sur un ViT pré-entraîné à grande échelle. Sans un tel point de départ, il n'y a rien pour compenser. Le modèle est lourd : 94 millions de paramètres, contre 41 pour la baseline. L'attention est quadratique : en pleine résolution, il y aurait quatre fois plus de patchs, et environ seize fois plus de calcul d'attention. Sa vitesse n'existe que sur GPU. Et les petits objets restent les plus faibles ; sur les panneaux, je suppose que le pré-entraînement COCO joue, mais c'est une hypothèse.

Les pistes : d'abord la résolution native, le levier principal pour les petits objets. Un test préalable avec un Mask2Former entraîné sur Cityscapes passe de 0,708 à 0,742 sur dix images. Ensuite, l'augmentation de données de l'article, un entraînement plus long, un ViT plus grand. Et, sans ré-entraîner, l'augmentation au moment du test.

Cumul : 13:10

19. Limites de l'étude et suites **0:30**

Les limites de l'étude. La validation sert à la fois à choisir la meilleure époque et à mesurer : les scores sont un peu optimistes, mais de la même façon pour les trois modèles. Aucun modèle n'a d'augmentation de données, et on observe un sur-apprentissage modéré. Les points de départ pré-entraînés diffèrent, c'est inhérent à chaque méthode.

Suites : plusieurs entraînements par modèle pour mesurer la variance, servir EoMT par l'API sur une machine GPU, le quantifier pour le CPU, et l'évaluer sur le jeu de test officiel.

Cumul : 13:40

Partie 3 · Démonstration et conclusion (≈ 5 min)

20. Le dashboard, déployé sur le cloud **4:30 (en direct)**

Avant la présentation : ouvrir le dashboard quelques minutes à l'avance pour le réveiller (Space gratuit). Garder la capture de cette diapositive comme secours en cas de problème réseau.

Je passe maintenant au dashboard. Il est écrit avec Streamlit et déployé sur un Space Hugging Face, avec les trois modèles exécutés sur CPU. Il a trois pages.

Page Dataset (≈ 1 min 15). Ici, l'analyse exploratoire du jeu de données : le nombre d'images par ville, et la part de chaque classe. *(Choisir « bicycle » dans la galerie.)* La galerie montre des exemples par classe : voici des vélos, souvent petits et partiellement cachés. *(Choisir CLAHE, bouger le curseur.)* On peut aussi appliquer une transformation, par exemple l'égalisation adaptative CLAHE, et comparer l'histogramme avant et après.

Page Prediction (≈ 2 min). *(Choisir une image, modèle EoMT, cliquer Predict.)* Je choisis une image de validation et le modèle EoMT. Voici le masque prédit, à côté du vrai masque, et le tableau des scores par classe pour cette image. *(Activer « Compare all models ».)* En activant la comparaison, les trois modèles tournent sur la même image. Sur CPU, cela prend une dizaine de secondes. *(Pendant l'attente :)* comme on l'a vu, sans GPU l'avantage de vitesse d'EoMT disparaît. *(Résultat affiché :)* on retrouve les taches parasites de la baseline, et les contours plus propres des transformers.

Page Model comparison (≈ 45 s). Enfin, la synthèse : le graphique précision contre vitesse, et les intervalles du bootstrap.

Accessibilité (≈ 30 s). Le dashboard suit les critères WCAG : chaque image a un texte alternatif, chaque graphique est doublé d'un tableau, les couleurs restent lisibles pour les daltoniens, et tout se commande au clavier.

Cumul : 18:10

21. Conclusion **0:30**

Pour conclure. EoMT, un modèle de 2025, bat la baseline de 2,7 points de mIoU, de façon statistiquement significative : 15 % de l'erreur restante en moins, et davantage sur les classes difficiles. Il égale Mask2Former en précision, en étant 2,4 fois plus rapide sur GPU.

Ma recommandation : adopter EoMT pour la prochaine version de CityVision, servi sur GPU, avec de l'augmentation de données et la résolution native.

Merci. Je suis à votre disposition pour les questions.

Cumul : 18:40 · marge ≈ 1 min 20

Questions probables et éléments de réponse

Le gain de 0,027 n'est-il pas faible ? À 0,82 de mIoU, chaque point coûte cher. EoMT supprime 15 % de l'erreur restante, 29 % sur le fond, et le bootstrap montre que le gain est toujours positif (intervalle à 95 % : 0,022 à 0,032).

Pourquoi EoMT plutôt que Mask2Former, puisqu'ils sont à égalité ? Même précision, mais 2,4 fois plus rapide sur GPU (43 ms contre 106 ms), et une architecture plus simple : un seul ViT, sans décodeur.

La comparaison est-elle équitable, avec des pré-entraînements différents ? Tout ce qui pouvait être identique l'est : données, résolution, échantillonneur, budget, métrique, code d'évaluation. Aucun modèle n'avait vu Cityscapes. Le pré-entraînement fait partie de chaque méthode.

Pourquoi pas d'augmentation de données ? Pour que les trois modèles soient comparés dans les mêmes conditions que la baseline existante, qui n'en avait pas. C'est la première amélioration proposée.

Pourquoi 512 × 1024 et pas la résolution native ? C'est la résolution du projet et de la baseline, et la contrainte des 8 Go de GPU. La résolution native est la piste principale d'amélioration.

Pourquoi le dashboard est-il lent ? Il tourne sur 2 CPU gratuits, sans GPU ni demi-précision : environ 1,6 s, 3,2 s et 4,1 s par image. En production, EoMT serait servi sur GPU.

Les résultats sont-ils reproductibles ? Oui : les trois modèles ré-entraînés de zéro dans le notebook livré donnent des mIoU à moins de 0,002 des entraînements de référence.

Pourquoi le ciel baisse-t-il ? C'est une grande zone homogène, déjà à 0,918 pour la baseline. La baisse est faible ($-0{,}011$) ; le gain des transformers porte sur les objets.